

ROS2 Spring Workshop

Lab 4: The Seeder Node (Services & Clients)

Project Context: The Triathlon Seeder

In **Phase 1** of your Final Project, your tractor drives through a field, and at specific locations it must “plant” a seed. Dropping a marker that other code can later verify.

How does your controller tell the simulator “plant a seed HERE”? Not with a Topic, those are for continuous streams like GPS or velocity. You need a one-shot, confirmable transaction: “Do this specific thing. Tell me when it’s done. Tell me if it failed.”

That’s a **Service**. Today you build the seeder that will drive your Phase 1 score.

Objective

In previous labs, we used Topics to stream data like a radio broadcast. Topics are terrible for things that must happen exactly once. If you publish “spawn a seed here!” on a Topic, your seed might get dropped, duplicated, or lost in traffic.

In this lab, you use Services to perform specific, one-time transactions.

You will learn:

- **Service Clients:** How to write code that asks another node to do a job.
- **The Handshake:** Understanding the Request/Response cycle.
- **Asynchronous Calls:** How to order food without freezing your robot’s brain.
- **Sim vs. Reality:** Why “the obvious way” works in simulation but breaks on real hardware, and how to write defensive code from day one.

Pre-Lab Concept Primer: Radio vs. SkipTheDishes

1. Topics (The Radio Broadcast)

Think of a Topic as a live radio station.

- **The Publisher (DJ):** They talk continuously. They don’t care if you are listening.
- **The Data:** If you tune in late, you missed the song. It’s gone forever.
- **Use Case:** High-speed data like GPS or wheel speeds. We don’t care about the packet from 1 second ago; we only want the newest one.

2. Services (SkipTheDishes)

Think of a Service as using a food delivery app. This is a **transaction**.

- **The Request (Place Order):** You select your items and hit “Checkout” (you send specific data).
- **The Process (Tracking):** The restaurant has to accept the order and cook it. This takes time!
- **The Response (Delivery):** The driver arrives and hands you the food (success) OR the app says “Restaurant cancelled” (failure).
- **Use Case:** Spawning seeds, resetting the simulation, taking a photo, triggering the sprayer. You need to know if it actually happened!

Feature	Topic (Radio)	Service (SkipTheDishes)
Communication	One-way (fire & forget)	Two-way (request & response)
Timing	Continuous (e.g., 30 Hz)	Discrete (event-based)
Reliability	Low (dropped packets OK)	High (must confirm success)

Table 1: The fundamental difference in ROS communication patterns.

3. The “Async” Revolution

This is the hardest concept in this lab. When we send a request, we have to wait for the answer. **How** we wait matters.

The Delivery Analogy

Scenario A: Synchronous (Blocking), “The Landline Call”

You call the restaurant. They put you on hold. You sit there holding the phone to your ear for 10 minutes. You cannot do anything else (like drive or work) because you are glued to the phone waiting for them to pick up.

Robot effect: the robot stops driving while waiting for the calculation.

Scenario B: Asynchronous (Non-Blocking), “The App”

You tap “Order” on SkipTheDishes. The app gives you a **tracking screen** (a “Future” object). You put your phone in your pocket and go back to playing video games. You only check the door when the app pings you.

Robot effect: the robot keeps driving while the calculation happens in the background.

Part 1: Package Setup

1. Navigate to the source folder:

```
1 cd /workspaces/agtech_ros2/src
2
```

2. Create a new package:

```
1 ros2 pkg create --build-type ament_python lab4_services
2
```

3. Verify the structure: ensure the folder `lab4_services` was created.

Part 2: Manual Service Calls (Play God in the Terminal)

Before writing code, let’s play God. We need to inspect the “interface” to know what data the service expects.

1. Run the simulator:

```
1 ros2 run turtlesim turtlesim_node
2
```

2. Check the service type. We need to know what “form” to fill out to use `/spawn`.

```
1 ros2 service type /spawn
2 # Output: turtlesim/srv/Spawn
3
```

3. Inspect the form:

```
1 ros2 interface show turtlesim/srv/Spawn
2
```

Output interpretation:

- Above the dashes (---): the **Request** (x, y, theta, name).
- Below the dashes: the **Response** (name).

4. Call the service manually. Try to spawn a seed at $x = 2, y = 2$:

```
1 ros2 service call /spawn turtlesim/srv/Spawn \
2 "{x: 2.0, y: 2.0, theta: 0.0, name: 'seed_1'}"
3
```

Syntax Spotlight: Anatomy of a Service Call

This command is complex; let's break it down:

- `ros2 service call`: The verb (initiate a transaction).
- `/spawn`: The **service name** (the specific phone number we are dialing).
- `turtlesim/srv/Spawn`: The **service type** (the language we must speak).
- `"{x: 2.0, ...}"`: The **request data**. This is written in **YAML** (Yet Another Markup Language).
 - It acts like a Python dictionary.
 - **Critical**: You must use double quotes " around the whole block and a space after every colon :.

Part 3: The Seeder Script

Writing a Service Client is complex. To save time, the logic has been pre-written in the lab resources. Your job is to install it and analyze it.

1. Copy the script into your package:

```
1 cp /workspaces/agtech_ros2/lab_resources/lab4/spawn_seed.py \
2   /workspaces/agtech_ros2/src/lab4_services/lab4_services/
3
```

2. Open the file for analysis:

```
1 code /workspaces/agtech_ros2/src/lab4_services/lab4_services/spawn_seed.py
2
```

Code Analysis: Read the code in your editor. Note the key components:

```
1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 from turtlesim.srv import Spawn # IMPORT THE SERVICE TYPE
5
6 class Seeder(Node):
7     def __init__(self):
8         super().__init__('seeder')
9
10    # 1. CREATE CLIENT: Open the App
11    self.client = self.create_client(Spawn, '/spawn')
12
13    # 2. CHECK FOR SERVER: Is the simulator running?
14    while not self.client.wait_for_service(timeout_sec=1.0):
15        self.get_logger().info('Waiting for turtlesim /spawn service...')
16
17    # 3. PREPARE THE ORDER (reusable request object)
18    self.req = Spawn.Request()
19
20    def plant(self, x, y, name):
21        # Fill out the order form
22        self.req.x = float(x)
23        self.req.y = float(y)
24        self.req.theta = 0.0
25        self.req.name = name
26
```

```

27     # ASYNC CALL: Returns a Future (tracking number) instantly
28     future = self.client.call_async(self.req)
29
30     # WAIT FOR DELIVERY: Block until the simulator confirms
31     rclpy.spin_until_future_complete(self, future)
32     return future.result()
33
34 def main(args=None):
35     rclpy.init(args=args)
36     node = Seeder()
37
38     # Plant a short row of 3 seeds across the middle of the field
39     for i in range(1, 4):
40         node.get_logger().info(f"Planting seed {i}...")
41         try:
42             # Calculate x to space them out: (2, 4, 6)
43             response = node.plant(float(i*2), 5.0, f"seed_{i}")
44             node.get_logger().info(f"Success! Created: {response.name}")
45         except Exception as e:
46             node.get_logger().error(f"Failed: {e}")
47
48     node.destroy_node()
49     rclpy.shutdown()
50
51 if __name__ == '__main__':
52     main()

```

Listing 1: spawn_seed.py (Reference Only)

Code Walkthrough: Chunk by Chunk

Let's break down exactly what this script is doing:

- **Lines 1–4 (The Shebang & Imports):** We start with the standard Linux shebang and our usual ROS 2 imports (`rclpy`, `Node`). However, notice the critical difference from Lab 3: instead of importing a *message* type like `Twist` or `Pose`, we import a *service* type: `from turtlesim.srv import Spawn`. Service types define both the request (what we send) and the response (what we get back), bundled into one object.
- **Lines 6–18 (The Constructor — Opening the App):** The `__init__` method sets up three things. First, `create_client(Spawn, '/spawn')` opens a connection to the `/spawn` service, like installing a delivery app on your phone. Second, the `while not self.client.wait_for_service()` loop checks whether the simulator is actually running and accepting orders; if not, the node waits patiently and logs a message every second instead of crashing. Third, `self.req = Spawn.Request()` creates a blank order form that we can fill out and reuse each time we want to plant a seed.
- **Lines 20–30 (The Plant Method — Placing an Order):** The `plant()` function is the core transaction. Lines 22–25 fill in the order form with the coordinates and name we want. Line 28 calls `call_async()`, which is like tapping “Place Order” on SkipTheDishes: it sends the request and immediately hands us back a **Future** (a tracking number), so our code isn't frozen waiting. Line 31 then calls `spin_until_future_complete()`, which tells ROS to keep processing events until the response arrives. Once it does, we return the result. This two-step pattern (fire async, then wait) is safe here because we are calling from `main()`, not from inside a callback.
- **Lines 32–49 (The Engine — Running the Planting Mission):** The `main()` function initializes ROS and creates the node, then runs a simple `for` loop to plant three seeds spaced 2m apart across the middle of the field. Notice the `try/except` block wrapping each `plant()` call: if a seed name already exists (because you ran the script twice), the service will raise an error. Instead of crashing the whole program, we log a warning and move on to the next seed. This defensive pattern becomes critical in your Final Project, where a single crash means a failed run. Finally, we clean up with `destroy_node()` and `rclpy.shutdown()`, just like in every previous lab.

Syntax Spotlight: The Client

- `create_client(Type, Name)`: Opens the app connection.
- `wait_for_service()`: Checks if the restaurant is accepting orders.
- `call_async(req)`: Hits the “Place Order” button. It returns a **Future** (tracking number) instantly, so your code doesn’t freeze.

Critical Warning: Do NOT mix Lab 3 and Lab 4

The trap: In Lab 3, you learned to use **callbacks** (the doorbell) to react to sensors. You might be tempted to put this service call inside your callback. For example, “if I see a weed, spawn a marker.”

Why this fails (deadlock):

- ROS 2 nodes run on a **single-threaded executor** by default. There is one “brain” processing one event at a time.
- `spin_until_future_complete()` works by telling the executor to keep spinning until the response arrives.
- But if you’re already *inside* a callback, the executor is currently busy running *your callback*. It can’t pick up the response while it’s stuck waiting on you.
- **Result:** the brain waits for the response, the response can’t be delivered until the brain is free, and the robot freezes forever. (Part 6’s stretch goal shows the right pattern: `add_done_callback()`.)

Rule: Only call services from the **main** loop or using special “Action Servers” (covered in Phase 2).

Part 4: Registration & Build

1. **Edit `setup.py`.** Add the entry point so ROS can find your script.

```
1 entry_points={
2     'console_scripts': [
3         'spawn_seed = lab4_services.spawn_seed:main',
4     ],
5 },
6
```

2. **Build:**

```
1 cd /workspaces/agtech_ros2
2 colcon build --symlink-install
3 source install/setup.bash
4
```

3. **Run it:**

```
1 ros2 run lab4_services spawn_seed
2
```

You should see three little turtles appear in a row across the middle of the simulator. Congrats!! You just used a service to modify the simulation world.

Summary: The ROS 2 Toolkit (Cheat Sheet)

By now, you have learned the “big three” tools for making robots think. Use this table to decide which one to reach for.

Tool	Analogy	Control Direction	When to use?
Timer (Lab 2)	Heartbeat	Internal (clock triggers code)	Periodic tasks: “Check battery every 1 s.” “Calculate speed at 100 Hz.”
Subscriber (Lab 3)	Doorbell	Inbound (world triggers code)	Reactions: “If GPS says $X > 9$, stop.” “If camera sees red, stop.”
Service Client (Lab 4)	SkipTheDishes	Outbound (code triggers world)	Transactions: “Plant a seed here.” “Reset the simulation.” “Calibrate the camera.”

Part 5: Challenge, The Precision Seeder

AgTech Context: Real automated seeders don’t plant randomly. The tractor drives across the field in a grid pattern, dropping seeds at precise intervals to optimize plant density and yield.

Your job: write a new script, `seed_field.py`, that plants a **3 row × 4 column grid of wheat** at 2 m spacing. Twelve seeds total.

You’ll also confront a real-world bug class: the **simulator-reality gap**, where code that works fine in sim falls apart on real hardware.

Part 5.1: The Naive Version

Start simple. Nested loops, plant a seed at each grid point, done:

```

1 # Inside your main function - the naive version
2 start_x, start_y = 1.0, 1.0
3 spacing = 2.0
4
5 for row in range(3):
6     y = start_y + row * spacing
7     for col in range(4):
8         x = start_x + col * spacing
9         try:
10            node.plant(x, y, f"wheat_{row}_{col}")
11        except Exception as e:
12            node.get_logger().warn(f"Plant failed at ({x},{y}): {e}")

```

Write this, build, run it. Then count your wheat.

Count Your Seeds With CLI

How many turtles actually made it into the world? Use the service list to count:

```
ros2 service list | grep wheat | wc -l
```

Each turtle that spawns successfully registers 3 services. If the count equals $12 \times 3 = 36$, all 12 planted.

Part 5.2: The Pitfall (Sim Forgiveness vs. Real-World Reality)

Look closely at what your script just did. The naive version probably “works” in turtlesim, you saw 12 seeds spawn, the count returns 36. Job done?

Not on real hardware. Here’s what the naive code is actually doing every iteration:

1. Tractor sits still at its current location.
2. Code asks the simulator: “Spawn a seed at coordinates (x,y) .”
3. Simulator obliges — it doesn’t check whether the tractor is in the way, because turtlesim has no collision physics.

A real automated seeder cannot do this. The seed dispenser is rigidly mounted on the chassis at a fixed offset from the tractor itself. If the tractor's wheels are at (x, y) , you literally cannot drop a seed at (x, y) — the dispenser is on the back of the chassis, 1 m away, and the seed goes there.

This is the **simulator-reality gap**: code that works in sim because the simulator lacks the physics that would expose the bug. We saw a version of this in Lab 3 (angle wraparound in the headland turn); we'll see it again in Lab 8 (Ackermann steering on the real tractor).

The lesson: write the right pattern from the start, before the bad habit becomes a real-hardware bug.

Part 5.3: The Fix (Step Aside, Then Plant)

The right pattern is to model what the real tractor would have to do: move into a position where the dispenser ends up over the target spot, *then* drop the seed.

In our 2D simulation we'll simplify this to: shift the tractor 1 m to the right of the target, then plant at the target. (On a real seeder you'd compute the dispenser offset from the tractor's pose; here, 1 m is good enough.)

The corrected sequence for each grid point:

1. Teleport `turtle1` 1 m to the RIGHT of the target position.
2. Call `/spawn` to plant the seed at the target position.

You'll need a **second service client** on your node — one for `/spawn` (already there) and one for `/turtle1/teleport_absolute`. The pattern is exactly the same as in Part 3, just with a different service type.

Hint: Find the Teleport Service

```
ros2 service list | grep turtle1
ros2 service type /turtle1/teleport_absolute
ros2 interface show turtlesim/srv/TeleportAbsolute
```

The interface has three fields: `x`, `y`, `theta`. Fill them in just like you did for `Spawn`.

Part 5.4: The Requirements

Your final `seed_field.py` must:

1. Spawn exactly **12 seeds** in a 3×4 grid, spacing 2 m.
2. Move `turtle1` aside before each plant.
3. Name each seed `wheat_{row}_{col}` so you can verify them later.
4. Wrap each `plant()` call in `try/except` so re-running your script doesn't crash when names collide.

Verifying Your Field

After the script runs, count the seeds:

```
ros2 service list | grep wheat | wc -l
```

You should get a number that is a multiple of 3 (each turtle registers 3 services: `pose`, `color_sensor`, etc.). If the count equals $12 \times 3 = 36$, all 12 planted.

Part 6: Stretch Goal, The Auto-Planter

(Optional, but combines everything you've learned this semester.)

Instead of a pre-programmed grid, drive the tractor yourself and have the controller drop a seed every few seconds *automatically* while you drive. This is closer to how a real seeder actually works, you don't manually trigger every plant; the controller schedules them at fixed intervals while the tractor moves.

The setup:

- Terminal 1: `turtlesim_node`

- Terminal 2: `ros2 run turtlesim turtle_teleop_key`
- Terminal 3: your auto-planter script

The design: A single node that combines all three patterns from this semester:

- **Subscriber** (Lab 3) on `/turtle1/pose`, updates `self.current_x`, `self.current_y` as you drive.
- **Timer** (Lab 2) at 0.33 Hz (every 3 seconds), fires the planting logic.
- **Service Client** (Lab 4) on `/spawn`, plants a seed at the most recent pose.

Drive the tractor in a zigzag for 30 seconds. You should see seeds appear in a trail behind the tractor at 3-second intervals.

A Small Twist on the Service Pattern

In Part 3, your `plant()` method used `rclpy.spin_until_future_complete()` to wait for the response inline. **You cannot do this inside a timer callback** — for the same reason the Critical Warning earlier in this lab told you not to wait inside any callback. The single-threaded executor would deadlock.

Instead, fire the request and tell ROS what to do when the response arrives:

```
def timer_callback(self):
    if self.current_x is None:
        return # haven't received a pose message yet

    self.spawn_req.x = self.current_x
    self.spawn_req.y = self.current_y
    self.spawn_req.theta = 0.0
    self.spawn_req.name = f"auto_seed_{self.count}"

    future = self.spawn_client.call_async(self.spawn_req)
    future.add_done_callback(self.on_spawn_done)
    self.count += 1

def on_spawn_done(self, future):
    response = future.result()
    self.get_logger().info(f"Planted {response.name}")
```

The timer callback fires the request and returns immediately. When the response eventually arrives, ROS calls `on_spawn_done` automatically. No waiting, no deadlock.

Why This Design Works

All three patterns run on the same single-threaded executor (the one started by `rclpy.spin`). The executor naturally interleaves them:

- The subscriber callback fires whenever a pose message arrives, keeping `self.current_x/y` fresh.
- The timer callback fires every 3 seconds, builds a request from the latest pose, and dispatches it asynchronously.
- The done-callback fires whenever the spawn service confirms.

No threading, no `input()`, no blocking. This is exactly how a real autonomous seeder is structured: a sense-decide-act loop where each piece runs when it's ready, not when it's blocked.

Challenge goals:

- Drive the tractor in a zigzag for ~30 seconds. Confirm 10 seeds appear at 3-second intervals.
- Add a counter; log "Seed N planted at (x,y)" on each successful spawn.
- Bonus: skip planting when the tractor isn't moving (compare consecutive poses; skip if movement is below a small threshold). This stops you from piling seeds on top of each other when you pause.

Deliverables

Submit a PDF containing:

1. **Manual spawn:** Terminal output of your `ros2 service call /spawn ...` from Part 2.
2. **Grid screenshot:** A screenshot of turtlesim after your `seed_field.py` has completed, showing all 12 seeds in a grid pattern.
3. **Count proof:** Output of `ros2 service list | grep wheat | wc -l` showing 36 (12 seeds $\times$ 3 services each).
4. **Reflection (3–5 sentences):** Your naive grid planter “worked” in turtlesim, but it would fail catastrophically on a real seeder. In your own words, explain why turtlesim hides this bug, why a real seeder would expose it, and what the “step aside, then plant” pattern is doing differently. Then name one other example of the simulator-reality gap you can imagine in agricultural robotics.
5. **(Optional) Stretch goal proof:** Screenshot or video of the auto-planter from Part 6, showing a trail of seeds appearing as you drove, plus the terminal output of at least 5 “Planted auto_seed_N” log lines.

What’s Next: Lab 5

You’ve now built a publisher (Lab 2), a subscriber (Lab 3), and a service client (Lab 4) — the three core ROS communication patterns. All in simulation, all reading perfect `/turtle1/pose` data piped straight from turtlesim.

Real robots don’t have that luxury. Their sensors lie in beautiful, mathematically structured ways, and your code has to compensate.

In Lab 5 you’ll build your tractor’s first real sensor: a webcam + AprilTag detector that computes the 3D pose of a printed marker. You’ll also confront **camera calibration** (turning a \$20 webcam into a measurement instrument) and the **transform tree** (how ROS tracks “where is X relative to Y?”). It’s the first lab where your code talks to physical hardware.